

گزارشکار آزمایش سوم سیستم عامل



پارسا احمدی ناو، آریا عازم، مجید صادقی نژاد

پرسش ها

- پرسش 1 :

ساختار PCB در xv6 درون فایل proc.h قرار گرفته و در struct مربوط به proc برای هر پردازش مشاهده می شود که مطابق زیر است :

```
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this
process
    enum procstate state;   // Process state
    int pid;                // Process ID
    struct proc *parent;    // Parent process
    struct trapframe *tf;   // Trap frame for current syscall
    struct context *context; // swtch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;             // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];           // Process name (debugging)
    int logged_in_user;      // if no ones logged in, value is -1
};

// Process memory is laid out contiguously, low addresses first:
//  text
//  original data and bss
//  fixed-size stack
//  expandable heap
```

به ازای هر پروسه در xv6 شش وضعیت اصلی تعریف شده است که مطابق زیر می باشند :

```
enum procstate {UNUSED, EMBYRO, SLEEPING, RUNNABLE, RUNNING,
ZOMBIE };
```

شباهت ها :

- 1: enum procstate state == process state
- 2: int pid == process number
- 3: struct context *context -> eip == program counter
- 4: uint sz == memory limit
- 5: struct file *ofile[NOFILE] == list of open files

- پرسش 2:

وضعیت های شش گانه اصلی پیشتر ذکر شدند. به طور کلی ، داریم :

UNUSED :

در شکل معادلی ندارد

EMBYRO :

در شکل ، معادل new است. فضای لازم برای PCB تخصیص یافته است اما pid ندارد.

SLEEPING :

در شکل معادل waiting است. پردازش منتظر یک رویداد (event) است.

RUNNABLE :

در شکل معادل ready است. پردازش آماده اجراست و در صف قرار دارد.

RUNNING :

در شکل معادل running است. پردازش در حال اجرا است.

ZOMBIE :

در شکل معادل terminated است. پردازش خاتمه پیدا کرده است.

- پرسش 3 :

```

120 void
121 userinit(void)
122 {
123     cprintf("hehe");
124     struct proc *p;
125     extern char _binary_initcode_start[], _binary_initcode_size[];
126
127     p = allocproc();
128
129     initproc = p;
130     if((p->pgdir = setupkvm()) == 0)
131         panic("userinit: out of memory?");
132     inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
133     p->sz = PGSIZE;
134     memset(p->tf, 0, sizeof(*p->tf));
135     p->tf->cs = (SEG_UCODE << 3) | DPL_USER;
136     p->tf->ds = (SEG_UDATA << 3) | DPL_USER;
137     p->tf->es = p->tf->ds;
138     p->tf->ss = p->tf->ds;
139     p->tf->eflags = FL_IF;
140     p->tf->esp = PGSIZE;
141     p->tf->eip = 0; // beginning of initcode.S
142
143     safestrcpy(p->name, "initcode", sizeof(p->name));
144     p->cwd = namei("/");
145
146     acquire(&ptable.lock);
147
148     p->state = RUNNABLE;
149
150     release(&ptable.lock);
151     // I added:
152     userinit_extra();
153 }
```

```

180 fork(void)
181 {
182     int i, pid;
183     struct proc *np;
184     struct proc *curproc = myproc();
185     if((np = allocproc()) == 0){
186         return -1;
187     }
188     if((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0){
189         kfree(np->kstack);
190         np->kstack = 0;
191         np->state = UNUSED;
192         return -1;
193     }
194     np->sz = curproc->sz;
195     np->parent = curproc;
196     *np->tf = *curproc->tf;
197     np->tf->eax = 0;
198     for(i = 0; i < NOFILE; i++)
199         if(curproc->ofile[i])
200             np->ofile[i] = filedup(curproc->ofile[i]);
201     np->cwd = idup(curproc->cwd);
202     safestrcpy(np->name, curproc->name, sizeof(curproc->name));
203     pid = np->pid;
204     acquire(&ptable.lock);
205     np->state = RUNNABLE;
206     release(&ptable.lock);
207     return pid;
208 }
```

دو تابع `userinit` و `fork` در تغییر حالت ها نقش دارند : `userinit` در تغییر حالت در صورتی که پردازش `init` باشد و `fork` هنگامی که پردازش `init` نباشد. درون هر یک از توابع یک تابع `allocproc` داریم که یک جایگاه استفاده نشده را پیدا می کند و حالت آن را به `EMBRYO` تغییر می دهد ، به آن `kernel stack` می دهد و `process context` را مقدار دهی اولیه می کند. بعد از فراخوانی این تابع و انجام مقدار دهی های دیگر برای پردازش مربوطه ، در هر دو تابع ، بعد از تابع `acquire` که به `process table lock` دسترسی می دهد (به منظور رعایت `thread safety`) حالت پردازش به `RUNNABLE` تغییر داده می شود.

تابع `allocproc`

```

73 static struct proc*
74 allocproc(void)
75 {
76     struct proc *p;
77     char *sp;
78     acquire(&ptable.lock);
79     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
80         if(p->state == UNUSED)
81             goto found;
82     release(&ptable.lock);
83     return 0;
84 found:
85     p->state = EMBRYO;
86     p->pid = nextpid++;
87     p->logged_in_user = -1;
88     release(&ptable.lock);
89     if((p->kstack = kalloc()) == 0){
90         p->state = UNUSED;
91         return 0;
92     }
93     sp = p->kstack + KSTACKSIZE;
94     sp -= sizeof *p->tf;
95     p->tf = (struct trapframe*)sp;
96     sp -= 4;
97     *(uint*)sp = (uint)trapret;
98     sp -= sizeof *p->context;
99     p->context = (struct context*)sp;
100     memset(p->context, 0, sizeof *p->context);
101     p->context->eip = (uint)forkret;
102     return p;
103 }

```

- پرسش 4 :

```
#define NPROC 64 // maximum number of processes
```

در فایل `param.h` می توان مشاهده کرد که حداکثر تعداد پردازش های ممکن توسط ثابت عمومی `NPROC` تعریف شده و برابر با 64 می باشد. مطابق کدی که از `allocproc` به اشتراک گذاشته شده ، هنگامی که پردازش ای سعی می کند بیش از حد مجاز فرزند ایجاد کند ، اتفاقی که رخ می دهد این است که سیستم عامل نمی تواند فضایی پیدا کند که حالتش `UNUSED` یا به عبارتی خالی باشد. در واقع جدول پردازش ها پر شده است و نمی توان جای جدیدی در آن به پردازش فرزند جدید تخصیص داد. با دقت در کد می بینیم که در `allocproc` ، یک حلقه مطابق زیر داریم :

```
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
```

مطابق این حلقه ، به اندازه محدودیت تعریف شده پیش می رویم تا یک جایگاه `UNUSED` را پیدا کنیم. در صورت یافتن ، به `found` می رویم و دستورات مطابق را اجرا می کنیم. در غیر این صورت ، صفر را بر می گردانیم. در صورتی هم که صفر را برگردانیم با توجه بخشی در `fork` که این اتفاق را پیش بینی کرده ، 1- را بر می گردانیم که به منزله خطا است :

```

if((np = allocproc()) == 0){
    return -1;
} }

```

- پرسش 5 :

کد مربوط به بخش scheduler را در زیر مشاهده می کنید :

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        sti();

        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;

            c->proc = p;
            switchvm(p);
            p->state = RUNNING;

            swtch(&(c->scheduler), p->context);
            switchkvm();

            c->proc = 0;
        }
        release(&ptable.lock);
    }
}
```

همان طور که مشاهده می شود ، حلقه این تابع به ازای هر بار اجرا ، پس از فراخوانی acquire ، بررسی می کند که آیا پردازهای در حالت RUNNABLE داریم یا خیر. در همین هنگام ، ممکن است بخش های دیگر کرنل مانند sleep/wakeup ، fork و یا exit وضعیت این پردازها را تغییر دهند. بنابراین لازم است که ptable را قفل کنیم چون در غیر این race condition رخ می دهد.

حتی در صورتی که سیستم تک هسته ای نیز باشد ، باز هم قفل کردن اهمیت دارد. در شرایطی که یک هسته داشته باشیم ، interrupt ها می توانند فراخوانی شده و باعث ایجاد تغییر در داده های پردازها بشوند. همچنین ممکن است که این پردازنده یک هسته ای به طور همزمان از چند thread خود استفاده کند (SMT) که در این شرایط نیز ممکن است شاهد race condition هایی باشیم که دلیلی بر استفاده از lock ها می باشد.

- پرسش 6 :

کد scheduler را در سوال قبلی نوشتیم. دو حالت پیش می آید :

- 1: پیش از آن که حلقه داخلی به به پردازش مورد نظر برسد، به RUNNABLE تغییر وضعیت داده است.

- 2: پس از آن که حلقه داخلی به پردازش مورد نظر رسیده، به RUNNABLE تغییر وضعیت داده است. با توجه به قفل بودن ptable ، در هر دو صورت ، در iteration بعدی امکان schedule کردن پیدا می کند.

- پرسش 7:

رجیسترهای context مطابق زیر می باشند :

```
// Saved registers for kernel context switches.
// Don't need to save all the segment registers (%cs, etc),
// because they are constant across kernel contexts.
// Don't need to save %eax, %ecx, %edx, because the
// x86 convention is that the caller has saved them.
// Contexts are stored at the bottom of the stack they
// describe; the stack pointer is the address of the context.
// The layout of the context matches the layout of the stack in swtch.S
// at the "Switch stacks" comment. Switch doesn't save eip explicitly,
// but it is on the stack and allocproc() manipulates it.
struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};
```

- پرسش 8:

رجیستر مربوطه در struct context برابر با uint eip می باشد. این رجیستر به کمک دستور call روی ptable قرار می گیرد و در آن ذخیره می شود و برخلاف سایر رجیستر ها لازم نیست که که push شود. همچنین به منظور بازیابی آن آن قدر popl می کنیم تا به انتهای پشته برسیم و در نهایت مقدار سر پشته را که همان program counter یا eip است را ret می کنیم :

```
1  .globl swtch
2  swtch:
3      movl 4(%esp), %eax
4      movl 8(%esp), %edx
5
6      pushl %ebp
7      pushl %ebx
8      pushl %esi
9      pushl %edi
10
11     movl %esp, (%eax)
12     movl %edx, %esp
13
14     popl %edi
15     popl %esi
16     popl %ebx
17     popl %ebp
18     ret
19
```

کد مربوط به swtch.S

- پرسش 9:

در صورت فراخوانی نکردن `sti()`، وقفه ها در حلقه scheduler غیرفعال باقی می مانند. این به معنای آن است که `ticks` هرگز افزایش پیدا نخواهند کرد. پس پردازش هایی که در حالت SLEEPING هستند هیچگاه بیدار نمی شوند چون به `ticks` وابسته هستند. پس توانایی `preemption` از کار می افتد و صرفا پردازش هایی که `yield()` را صریحا صدا می زنند کنار گذاشته می شوند. علاوه بر این، اگر تابعی در وضعیت RUNNING باشد، چون که هیچ `timer IRQ` نداریم، هیچگاه به `trap()` وارد نمی شویم تا `yield()` را بفرستیم مگر اینکه صراحتا دستور بدهیم. پس هیچ نقطه ای برای بازگشت به scheduler به صورت طبیعی وجود نخواهد داشت. همچنین چون که همه وقفه ها را قطع کرده ایم، تمامی I/O وابسته به وقفه ها بی اثر خواهند شد.

- پرسش 10:

به منظور پیدا کردن تعداد `interrupt` ها در هر ثانیه می توان به `lapic.c` مراجعه کرد. کد زیر از فایل مربوطه است:

```
lapicw(TDCR, X1);
lapicw(TIMER, PERIODIC | (T_IRQ0 + IRQ_TIMER));
lapicw(TICR, 10000000);
```

اتفاقی که در این کد رخ می دهد این است که ابتدا `timer` را با عدد 10,000,000 مقدار دهی می کنیم. سپس متناسب با فرکانس باس به پایین می شماریم تا به صفر برسیم. وقتی به صفر رسیدیم، `interrupt` می دهیم و مجدد شروع می کنیم. با توجه به اینکه فرکانس باس تقریبا 1GHz است، می توان به صورت تخمینی محاسبه کرد که به ازای هر ثانیه 100 تا `interrupt` خواهیم داشت. دقت شود که این مقدار تقریبی بوده و دقیق نیست. بخش زیر از کتاب نیز مقدار 100 تا `interrupt` به ازای هر ثانیه را گزارش می کند:

Code: Interrupts

Devices on the motherboard can generate interrupts, and xv6 must set up the hardware to handle these interrupts. Devices usually interrupt in order to tell the kernel that some hardware event has occurred, such as I/O completion. Interrupts are usually optional in the sense that the kernel could instead periodically check (or "poll") the device hardware to check for new events. Interrupts are preferable to polling if the events are relatively rare, so that polling would waste CPU time. Interrupt handling shares some of the code already needed for system calls and exceptions.

Interrupts are similar to system calls, except devices generate them at any time. There is hardware on the motherboard to signal the CPU when a device needs attention (e.g., the user has typed a character on the keyboard). We must program the device to generate an interrupt, and arrange that a CPU receives the interrupt.

Let's look at the timer device and timer interrupts. We would like the timer hardware to generate an interrupt, say, 100 times per second so that the kernel can track the passage of time and so the kernel can time-slice among multiple running processes. The choice of 100 times per second allows for decent interactive performance while not swamping the processor with handling interrupts.

- پرسش 11:

پس از رخ دادن timer interrupt ، پردازنده به تابع trap() می رود تا وضعیت را برطرف کند. در تابع trap() مطابق با این بخش ، این interrupt را handle می کنیم :

```
if(myproc() && myproc()->state == RUNNING &&
    tf->trapno == T_IRQ0+IRQ_TIMER)
    yield();
```

اتفاقی که رخ می دهد این است که ابتدا تشخیص می دهیم که interrupt رخ داده یک timer interrupt است و اینکه آیا پردازنده مورد نظر یک پردازنده در حالت RUNNING است یا خیر. سپس تابع yield() را فرا می خوانیم که باعث تغییر وضعیت از RUNNING به RUNNABLE می شود.

- پرسش 12:

با توجه به پرسش دهم و اینکه گفته شده که در هر یک ثانیه ، 100 تا interrupt خواهیم داشت ، می توانیم کوانتوم زمانی را مطابق زیر به دست بیاوریم :

$$Q = \frac{1}{\text{frequency of interrupts}} = \frac{1}{100} = 0.01 s = 10 ms$$

- پرسش 13:

کد تابع wait را در زیر مشاهده می کنید :

```
278 int
279 wait(void)
280 {
281     struct proc *p;
282     int havekids, pid;
283     struct proc *curproc = myproc();
284     acquire(&ptable.lock);
285     for(;;){
286         havekids = 0;
287         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
288             if(p->parent != curproc)
289                 continue;
290             havekids = 1;
291             if(p->state == ZOMBIE){
292                 pid = p->pid;
293                 kfree(p->kstack);
294                 p->kstack = 0;
295                 freevm(p->pgdir);
296                 p->pid = 0;
297                 p->parent = 0;
298                 p->name[0] = 0;
299                 p->killed = 0;
300                 p->state = UNUSED;
301                 release(&ptable.lock);
302                 return pid;
303             }
304         }
305         if(!havekids || curproc->killed){
306             release(&ptable.lock);
307             return -1;
308         }
309         sleep(curproc, &ptable.lock);
310     }
311 }
312
313 }
```

در انتهای این تابع مشاهده می کنید که از تابع sleep به صورت: sleep(curproc, &ptable.lock) استفاده شده است. اتفاقی که می افتد این است که wait بر روی اشاره گر به پردازنده جاری (curproc) می خوابد تا وقتی که پردازنده فرزندش تابع wakeup1 را فراخوانی کند.

- پرسش 14:

به طور کلی تابع `sleep` زمانی کاربرد دارد که می خواهیم شرطی را بررسی کنیم و همزمان قفل را نیز بر روی جدول پردازش ها نگه داریم : اگر شرط برقرار نباشد، `sleep` را فرا می خوانیم تا پردازش را متوقف کنیم و قفلش را به تنهایی (atomically) آزاد می کنیم. سپس وقتی با فراخوانی `wakeup` از `sleep` خارج می شویم، مجدد قفل ها را `acquire` می کنیم و شرط را بررسی می کنیم. مثال های متعددی از این کاربرد وجود دارد : استفاده در خواندن `pipe` های خالی یا نوشتن در `pipe` های پر ، عملیات I/O ، تخصیص منابعی که در حال حاضر در دسترس نیستند و مثال `pipe` را در زیر مشاهده می کنید :

```
if(p->nread == p->nwrite){
    sleep(&p->nread, &p->lock);
}
```

- پرسش 15:

مطابق توضیحاتی که پیشتر داده شد ، مسئولیت خارج کردن وضعیت پردازش از وضعیت خواب به عهده `wakeup` است که در زیر مشاهده می کنید :

```
static void
wakeup1(void *chan)
{
    struct proc *p;

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == SLEEPING && p->chan == chan)
            p->state = RUNNABLE;
}

void
wakeup(void *chan)
{
    acquire(&ptable.lock);
    wakeup1(chan);
    release(&ptable.lock);
}
```

مشاهده می کنید که تابع `wakeup1` وضعیت پردازش را ، اگر که در حالت `SLEEPING` باشد به `RUNNABLE` (که معادل همان `ready` است) تغییر می دهد.

- پرسش 16 :

همان طور که در کد نیز مشاهده می کنید ، وضعیت را از `SLEEPING` (معادل `waiting`) به وضعیت `RUNNABLE` (معادل `ready`) تغییر می دهد.

- پرسش 17 :

تابع kill را در زیر مشاهده می کنید :

```
int
kill(int pid)
{
    struct proc *p;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING)
                p->state = RUNNABLE;
            release(&ptable.lock);
            return 0;
        }
    }
    release(&ptable.lock);
    return -1;
}
```

در این تابع ، مشاهده می کنید که ابتدا بررسی می شود آیا وضعیت پردازش در حالت SLEEPING است ای خیر. اگر چنین بود ، وضعیت آن را به RUNNABLE تغییر می دهیم. دلیل این کار این است که به scheduler وارد شده و توسط scheduler ، مقدار kill =1 آن تشخیص داده شود تا فرآیند exit صورت گیرد. اگر از SLEEPING به RUNNABLE تغییر ندهیم ، kernel هرگز متوجه این تغییر نمی شود.

- پرسش 18 :

```
261 // Pass abandoned children to init.
262 for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
263     if(p->parent == curproc){
264         p->parent = initproc;
265         if(p->state == ZOMBIE)
266             wakeup1(initproc);
267     }
268 }
```

حلقه مقابل در تعریف تابع exit مشاهده می شود. اتفاقی که در این جا رخ می دهد این است که ابتدا تمامی فرزندان پردازش که خارج می شود ، به فرزندی initproc در می آیند. سپس بررسی می شود که به ازای هر کدام از این فرزندان ، آیا هیچ کدام از این پردازش ها در وضعیت ZOMBIE قرار

دارند یا خیر : اگر قرار داشتند ، initproc را بیدار می کنیم (wakeup1(initproc)) تا آن پردازش ها را حذف کند. دقت شود که initproc نیز پس از بیدار شدن ، در یک حلقه بی نهایتی قرار می گیرد که wait() را فرا می خواند تا وضعیت فرزندی که terminate شده اند را بیابد و فضای تخصیص یافته به آنها را آزاد کند. این فرآیند باعث می شود تا پردازش های orphan به زامبی های دائمی تبدیل نشوند و منابع آنها مورد استفاده مجدد قرار بگیرد.

پیاده سازی

- پرسش 19 :

با توجه این کلاس و سطح بندی و همان طور که گفته شده است ، shell در کلاس دوم قرار می گیرد. پس طبیعتاً با توجه به فرض اینکه تمام پردازنده ها مشغول به فعالیت های کلاس اول باشند ، scheduler هیچ گاه به کلاس دوم نمی رسد و هیچ task ای از کلاس دوم ، من جمله task های shell ، به scheduler نمی روند تا برای انجام شدن برنامه ریزی شوند. در عمل این به معنای این است که سیستم غیر قابل استفاده خواهد بود چون کاربر نمی تواند دستور جدیدی بدهد و هیچ گونه کاربری از سیستم دریافت نخواهد کرد. تنها راه ممکن این خواهد بود که تمامی پردازنده های کلاس اول به صورت اختیاری پردازنده را yield() کنند.

- پیاده سازی الگوریتم RR

نتیجه ی اجرای برنامه با یک پردازنده:

```
QEMU
Machine View
init: starting sh
Majid Sadeghinejad
Parsa Ahmadi
Aria Azem
majid-sadeghinejad@parsa-ahmadi:aria-azem $ rrttest
-- RR test --
pid 4 iter 0
pid 5 iter 0
pid 6 iter 0
pid 4 iter 1
pid 5 iter 1
pid 6 iter 1
pid 4 iter 2
pid 5 iter 2
pid 6 iter 2
pid 4 iter 3
pid 5 iter 3
pid 6 iter 3
pid 4 iter 4
pid 5 iter 4
pid 6 iter 4
majid-sadeghinejad@parsa-ahmadi:aria-azem $ _
```

نتیجه ی اجرای برنامه با دو پردازنده:

```
QEMU
Machine View
init: starting sh
Majid Sadeghinejad
Parsa Ahmadi
Aria Azem
majid-sadeghinejad@parsa-ahmadi:aria-azem $ rrttest
-- RR test --
pid 4 iter 0
pid 5 iter 0
pid 6 iter 0
pid 5 pid 4 iter 1
iter 1
pid 6 iter 1
pid 4 iter 2
pid 5 iter 2
pid 6 iter 2
pid 4 iter 3
pid 5 iter 3
pid 6 iter 3
pid 4 iter 4
pid 5 iter 4
pid 6 iter 4
majid-sadeghinejad@parsa-ahmadi:aria-azem $
```

- پرسش 20:

با افزایش CPUS ، دیگر ترتیب اجرا مشابه قبل نخواهد بود چون که در عمل دو پردازنده در کنار هم و به موازات یکدیگر عمل می کنند.

پیاده سازی FCFS:

از آن جا که تمام پردازش های جدید ساخته شده در سطح دوم کلاس دوم قرار دارند بنابراین همگی به صورت FCFS اجرا می شوند:

```
QEMU
Machine View
init: starting sh
Majid Sadeghinejad
Parsa Ahmadi
Aria Azem

majid-sadeghinejad@parsa-ahmadi:aria-azem$ schtest
-- SCHEDULE test --
pid 4 iter 0
pid 4 iter 1
pid 4 iter 2
pid 4 iter 3
pid 4 iter 4
pid 5 iter 0
pid 5 iter 1
pid 5 iter 2
pid 5 iter 3
pid 5 iter 4
pid 6 iter 0
pid 6 iter 1
pid 6 iter 2
pid 6 iter 3
pid 6 iter 4
majid-sadeghinejad@parsa-ahmadi:aria-azem$
```

پیاده سازی EDF:

```
QEMU
Machine View
Majid Sadeghinejad
Parsa Ahmadi
Aria Azem

majid-sadeghinejad@parsa-ahmadi:aria-azem$ schtest
Main process -> real-time, deadline: 1, pid: 3
New noraml process -> pid: 4
New noraml process -> pid: 5
New real-time process -> pid: 6, deadline: 20
New real-time process -> pid: 7, deadline: 3
EDF pid 7 iter 0
EDF pid 7 iter 1
EDF pid 7 iter 2
EDF pid 6 iter 0
EDF pid 6 iter 1
EDF pid 6 iter 2
FCFS pid 4 iter 0
FCFS pid 4 iter 1
FCFS pid 4 iter 2
FCFS pid 5 iter 0
FCFS pid 5 iter 1
FCFS pid 5 iter 2
majid-sadeghinejad@parsa-ahmadi:aria-azem$
```

- پرسش 21:

در صورتی که تسک ها Hard RT باشند ، الگوریتم EDF با فرض های زیر اجرا می شود :

- بدترین زمان اجرای هر پردازش و overhead آن محدود است و مقدارش را می دانیم
- CPU و سایر منابع مشترک به هنگام اجرا مدیریت می شوند.
- Timer interrupt ها هم می توانند به خوبی preempt و dispatch شوند به طوری که هیچ deadline را از دست ندهیم.
- پیش از اجرای برنامه ها یک تست برنامه ریزی انجام می دهیم تا مطمئن شویم که همه برنامه ها را می توانیم اجرا کنیم.

پیاده سازی aging و تست نهایی:

```
majid-sadeghinejad|parsa-ahmadi|aria-azem $ schtest
shell changed schedule class
pid: 13 from class 3 to class 2
User changed schedule class
pid: 13 from class 2 to class 1
Main process -> real-time, deadline: 1, pid: 13
New Noraml process (going to be Critical) -> pid: 14
New Noraml process (going to be Critical) -> pid: 15
New Noraml process (going to be Critical) -> pid: 16
New real-time process -> pid: 17, deadline: 100
New real-time process -> pid: 18, deadline: 3
New real-time process -> pid: 19, deadline: 20
REAL-TIME pid 18 iter 0
REAL-TIME pid 18 iter 1
REAL-TIME pid 18 iter 2
REAL-TIME pid 19 iter 0
pid: 14 from class 3 to class 2 (aging !)
pid: 15 from class 3 to class 2 (aging !)
pid: 16 from class 3 to class 2 (aging !)
REAL-TIME pid 19 iter 1
REAL-TIME pid 19 iter 2
REAL-TIME pid 17 iter 0
REAL-TIME pid 17 iter 1
```

```
REAL-TIME pid 17 iter 2
NORMAL pid 14 iter 0
NORMAL pid 15 iter 0
NORMAL pid 16 iter 0
NORMAL pid 14 iter 1
NORMAL pid 15 iter 1
NORMAL pid 16 iter 1
NORMAL pid 16 iter 2
NORMAL pid 14 iter 2
NORMAL pid 15 iter 2
New Noraml process -> pid: 20
New Noraml process -> pid: 21
New Noraml process -> pid: 22
NORMAL pid 20 iter 0
NORMAL pid 20 iter 1
NORMAL pid 20 iter 2
NORMAL pid 21 iter 0
NORMAL pid 21 iter 1
NORMAL pid 21 iter 2
NORMAL pid 22 iter 0
NORMAL pid 22 iter 1
NORMAL pid 22 iter 2
majid-sadeghinejad|parsa-ahmadi|aria-azem $
```

- پرسش 22:

در اکثر scheduler ها زمان انتظار به عنوان مدت زمانی تعریف می‌شود که یک پردازش در صف RUNNABLE قرار دارد و برای دستیابی به CPU رقابت می‌کند. هنگامی که یک پردازش در وضعیت SLEEPING است، واجد شرایط اجرا نیست – چون روی یک ورودی/خروجی (I/O) یا رویدادی مسدود شده – بنابراین:

- برای زمان CPU رقابت نمی‌کند
- جایگاه آن در صف آماده برای دیگر پردازش‌ها آزاد می‌شود
- در نظر گرفتن این زمان به عنوان زمان انتظار باعث افزایش مصنوعی معیارها (مثل زمان پاسخ‌گویی) می‌شود، در حالی که این زمان خارج از کنترل زمان‌بند است

- پرسش 23:

شما می‌خواهید شمارنده‌های "آماده" مربوط به هر کلاس (صف) را با هر جایی که یک پردازش را در وضعیت RUNNABLE یا RUNNING قرار می‌دهید، هماهنگ نگه دارید. به‌طور مشخص، در فایل `proc.c` باید:

1. شمارنده‌های خود را در ابتدا تعریف می‌کنیم.

2. پس از این خط کد :

```
p->state = RUNNABLE
```

(در توابعی مثل `userinit`, `fork`, `yield`, `wakeup1`, `kill`)

شمارنده‌ی مناسب را افزایش دهید.

3. قبل از این خط کد :

```
p->state = RUNNING
```

در تابع `scheduler` شمارنده‌ی مناسب را کاهش دهید.

پیاده سازی سیستم کال چاپ اطلاعات و تست نهایی تمام قابلیت ها

```
majid-sadeghinejad|parsa-ahmadi|aria-azem $ schtest
```

```
shell changed schedule class
```

```
pid: 3 from class 3 to class 2
```

```
User changed schedule class
```

```
pid: 3 from class 2 to class 1
```

```
Main process -> real-time, deadline: 1, pid: 3
```

```
New Noraml process (going to be Critical) -> pid: 4
```

```
New Noraml process (going to be Critical) -> pid: 5
```

```
New Noraml process (going to be Critical) -> pid: 6
```

```
New real-time process -> pid: 7, deadline: 100
```

```
New real-time process -> pid: 8, deadline: 3
```

```
New real-time process -> pid: 9, deadline: 20
```

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
2	SLEEP	NR	3	2	0	16	9
3	RUNNING	RT	1	0	1	316	45
4	RUNNBL	NR	3	29	0	332	0
5	RUNNBL	NR	3	23	0	338	0
6	RUNNBL	NR	3	18	0	343	0
7	RUNNBL	RT	1	0	450	350	0
8	RUNNBL	RT	1	0	358	355	0
9	RUNNBL	RT	1	0	380	360	0

REAL-TIME pid 8 iter 0

REAL-TIME pid 8 iter 1

REAL-TIME pid 8 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
2	SLEEP	NR	3	2	0	16	9
3	SLEEP	RT	1	0	1	316	46
4	RUNNBL	NR	3	718	0	332	0
5	RUNNBL	NR	3	712	0	338	0
6	RUNNBL	NR	3	707	0	343	0
7	RUNNBL	RT	1	0	450	350	0

8	RUNNING	RT	1	0	358	355	688
---	---------	----	---	---	-----	-----	-----

9	RUNNBL	RT	1	0	380	360	0
---	--------	----	---	---	-----	-----	---

REAL-TIME pid 9 iter 0

pid: 4 from class 3 to class 2 (aging !)

pid: 5 from class 3 to class 2 (aging !)

pid: 6 from class 3 to class 2 (aging !)

REAL-TIME pid 9 iter 1

REAL-TIME pid 9 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	47
---	-------	----	---	---	---	-----	----

4	RUNNBL	NR	2	800	0	332	0
---	--------	----	---	-----	---	-----	---

5	RUNNBL	NR	2	800	0	338	0
---	--------	----	---	-----	---	-----	---

6	RUNNBL	NR	2	800	0	343	0
---	--------	----	---	-----	---	-----	---

7	RUNNBL	RT	1	0	450	350	0
---	--------	----	---	---	-----	-----	---

9	RUNNING	RT	1	0	380	360	671
---	---------	----	---	---	-----	-----	-----

REAL-TIME pid 7 iter 0

REAL-TIME pid 7 iter 1

REAL-TIME pid 7 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
2	SLEEP	NR	3	2	0	16	9
3	SLEEP	RT	1	0	1	316	48
4	RUNNBL	NR	2	800	0	332	0
5	RUNNBL	NR	2	800	0	338	0
6	RUNNBL	NR	2	800	0	343	0
7	RUNNING	RT	1	0	450	350	603

NORMAL pid 4 iter 0

NORMAL pid 5 iter 0

NORMAL pid 6 iter 0

NORMAL pid 4 iter 1

NORMAL pid 5 iter 1

NORMAL pid 6 iter 1

NORMAL pid 4 iter 2

NORMAL pid 5 iter 2

NORMAL pid 6 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
2	SLEEP	NR	3	2	0	16	9
3	SLEEP	RT	1	0	1	316	49
4	RUNNING	NR	2	800	0	332	37

5	RUNNBL	NR	2	800	0	338	36
---	--------	----	---	-----	---	-----	----

6	RUNNBL	NR	2	800	0	343	36
---	--------	----	---	-----	---	-----	----

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	50
---	-------	----	---	---	---	-----	----

5	RUNNING	NR	2	800	0	338	38
---	---------	----	---	-----	---	-----	----

6	RUNNBL	NR	2	800	0	343	36
---	--------	----	---	-----	---	-----	----

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	50
---	-------	----	---	---	---	-----	----

5	RUNNBL	NR	2	800	0	338	39
---	--------	----	---	-----	---	-----	----

6	RUNNING	NR	2	800	0	343	38
---	---------	----	---	-----	---	-----	----

New Noraml process -> pid: 10

New Noraml process -> pid: 11

New Noraml process -> pid: 12

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	RUNNING	RT	1	0	1	316	61
---	---------	----	---	---	---	-----	----

10	RUNNBL	NR	3	8	0	2451	0
----	--------	----	---	---	---	------	---

11	RUNNBL	NR	3	4	0	2454	0
----	--------	----	---	---	---	------	---

12	RUNNBL	NR	3	1	0	2457	0
----	--------	----	---	---	---	------	---

NORMAL pid 10 iter 0

NORMAL pid 10 iter 1

NORMAL pid 10 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	62
---	-------	----	---	---	---	-----	----

10	RUNNING	NR	3	9	0	2451	38
----	---------	----	---	---	---	------	----

11	RUNNBL	NR	3	43	0	2454	0
----	--------	----	---	----	---	------	---

12	RUNNBL	NR	3	40	0	2457	0
----	--------	----	---	----	---	------	---

NORMAL pid 11 iter 0

NORMAL pid 11 iter 1

NORMAL pid 11 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	63
---	-------	----	---	---	---	-----	----

11	RUNNING	NR	3	45	0	2454	37
----	---------	----	---	----	---	------	----

12 RUNNBL NR 3 79 0 2457 0

NORMAL pid 12 iter 0

NORMAL pid 12 iter 1

NORMAL pid 12 iter 2

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	SLEEP	RT	1	0	1	316	64
---	-------	----	---	---	---	-----	----

12	RUNNING	NR	3	81	0	2457	38
----	---------	----	---	----	---	------	----

PID	STATE	TYPE	CLASS	WAIT	DL	ARR	CONS
-----	-------	------	-------	------	----	-----	------

1	SLEEP	NR	2	0	0	0	12
---	-------	----	---	---	---	---	----

2	SLEEP	NR	3	2	0	16	9
---	-------	----	---	---	---	----	---

3	RUNNING	RT	1	0	1	316	65
---	---------	----	---	---	---	-----	----

majid-sadeghinejad|parsa-ahmadi|aria-azem \$